Πολυτεχνείο Κρήτης
Σχολή ΗΜΜΥ
Δομές Δεδομένων και Αλγόριθμοι

Αλεξίου Σταμάτιος

Α.Μ: 2020030158

Παράδοση αναφοράς 2ης εργασίας
Ημερομηνία Παράδοσης: 06 Μαΐου 2023

Περιεχόμενα

1 Σύντομη περιγραφή του κώδικα **3**

2 Αποτελέσματα μετρήσεων και Κατασκευή διαγραμμάτων **4**

3 Τεκμηρίωση Αποτελεσμάτων **5**

1 Σύντομη περιγραφή του κώδικα

Ο κώδικας αποτελείται από 4 packages: το **counters**, το **kdtree** , το **quadtree** και το **tests**.[1]

- **counters** : Περιέχει τους μετρητές βάθους και χρόνου, οι οποίοι είναι υλοποιημένοι με την ίδια λογική του MultiCounter του φροντηστηρίου 2. Επιπλέον, έχουν προστεθεί οι συναρτήσεις averageDepthOnSearching και averageTimeOnSearching αντίστοιχα , οι οποίες επιστρέφουν το μέσο βάθος του δέντρου(με 2 δεκαδικά ψηφία) και το μέσο χρόνο αναζήτησης σε κάθε αναζήτηση.

- **kd-Tree** : Περιέχει τον κόμβο του Kd-Tree και το Kd-Tree. Στην πρώτη κλάση έχουμε τις μεταβλητές x,y που ουσιαστικά αποτελούν ένα σημείο (x,y) και επίσης κρατάμε σε μεταβλητές τον δεξί και τον αριστερό κόμβο γιατί θα μας βοηθήσει στην υλοποιήση του Kd-Tree. Στο Kd-Tree υλοποιήθηκαν οι μεθόδοι εισαγωγής και αναζήτησης στοιχείων με αναδρομή ελέγχοντας κάθε φορά το επίπεδο που βρισκόμαστε ,καθώς μας υποδεικνύει αν θα συγκρίνουμε ως προς το x ή το y . Μεγάλο κομμάτι κώδικα του Kd-Tree βρέθηκε από το διαδίκτυο και συγκεκριμένα από την ιστοσελίδα :
  `https://www.geeksforgeeks.org/search-and-insertion-in-k-dimensional-tree/`

- **quad-Tree** : Περιέχει τον κόμβο του Quad-Tree και το QuadTree. Στην πρώτη κλάση έχουμε τις μεταβλητές x,y που ουσιαστικά αποτελεί ένα σημείο (x,y) και επίσης κρατάμε σε μεταβλητές 4 ακόμη Qd κόμβους : τον nw , τον ne, τον sw και τον se γιατί θα μας βοηθήσει στην υλοποιήση του QuadTree. Στο QuadTree υλοποιήθηκαν οι μεθόδοι εισαγωγής και αναζήτησης στοιχείων με αναδρομή. Υπολογίζουμε κάθε φορά το μεσαίο σημείο και ανάλογα με τις συντεταγμένες του σημείου που θέλουμε να εισάγουμε/αναζητήσουμε βλέπουμε σε ποιον κόμβο θα πάμε(Στοιχεία εισάγονται μόνο σε φύλλα του δέντρου, που περικλείει το σημείο , ενώ κάθε φύλλο έχει το πολύ ένα μόνο στοιχείο). Η υλοποίηση του κώδικα βασίστηκε στη δομή του Kd-Tree , ενώ μέσω της ιστοσελίδας :
  `https://opendsa-server.cs.vt.edu/ODSA/Books/Everything/html/PRquadtree.html` , έγινε ευκολότερα αντιληπτό το πως ακριβώς γίνεται η εισαγωγή στοιχείων στο QuadTree (Μέσω του Interactive Visualization).

- **tests** : Περιέχει την κλάση Testings , η οποία περιέχει την main, μέσα στην οποία γίνονται όλες οι εισαγωγές και αναζητήσεις στα δύο δέντρα. Αρχικά, δημιουργούμε τα κλειδιά που θα εισάγουμε στα δέντρα. Επειδή, ο κωδικάς μας δημιουργεί τα x,y ξεχωριστά, για αριθμό κλειδιών μεγαλύτερο του N δεν μπορούμε να δημιουργήσουμε μοναδικά κλειδιά μέσω της συνάρτησης που χρησιμοποιούμε για μικρότερο αριθμό κλειδιών. Σε αυτή την περίπτωση, αποθηκεύουμε σε ένα set έναν συγκεκριμένο αριθμό(randX * M[i] + randY) ο οποίος είναι μοναδικός και δημιουργούμε τυχαία κλειδιά, με την προϋπόθεση ότι ο ακέραιος που δημιουργείται δεν υπάρχει ήδη στο set. Έτσι, εξασφαλίζουμε την μοναδικότητα των κλειδιών. Στην συνέχεια, πραγματοποιούμε τις αναζητήσεις και εκτυπώνουμε τα στατιστικά για κάθε δέντρο.

[1] Παραπάνω επεξηγήθηκαν συνοπτικά κυρίως τα πακέτα με τις κλάσεις που περιέχουν. Ωστόσο, στον κώδικα αναγράφονται σχόλια σχετικά με την υλοποίηση της κάθε συνάρτησης ξεχωριστά.

2 Αποτελέσματα μετρήσεων και Κατασκευή διαγραμμάτων

Τρέχοντας τον κώδικα παίρνουμε τα εξής αποτελέσματα τα οποία φαίνονται παρακάτω συγκεντρωμένα σε ένα πίνακα.

	Kd-Tree	Kd-Tree	QuadTree	QuadTree
M	Μέσος όρος βάθους δέντρου που φτάνουμε για επιτυχημένη αναζήτηση	Μέσος όρος βάθους δέντρου που φτάνουμε για αποτυχημένη αναζήτηση	Μέσος όρος βάθους δέντρου που φτάνουμε για επιτυχημένη αναζήτηση	Μέσος όρος βάθους δέντρου που φτάνουμε για αποτυχημένη αναζήτηση
200	8.83	10.87	4.61	5.24
500	10.06	12.55	5.34	6.02
1000	11.41	13.25	5.76	6.45
10000	15.97	17.96	7.46	8.11
30000	18.51	20.92	8.28	8.95
50000	20.31	21.65	8.6	9.24
70000	23.6	24.4	8.83	9.54
100000	20.57	22.54	9.24	9.79

Με βάση τον παραπάνω πίνακα δημιουργήσαμε τα 4 διαγράμματα τα οποία φαίνονται παρακάτω.

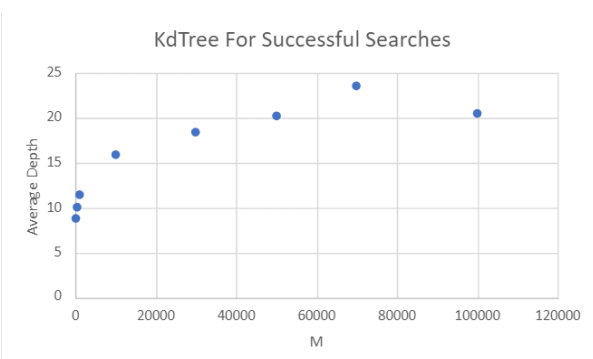

Figure 1: Diagram of Kd-Tree for successful searches

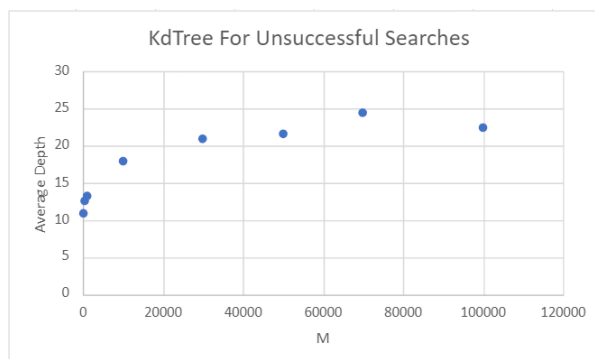

Figure 2: Diagram of Kd-Tree for unsuccessful searches

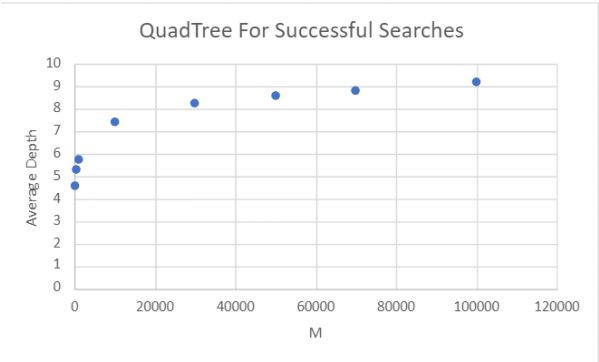

Figure 3: Diagram of QuadTree for successful searches

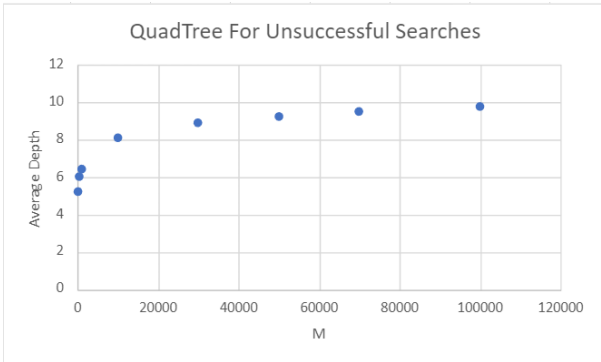

Figure 4: Diagram of QuadTree for unsuccessful searches

3 Τεκμηρίωση Αποτελεσμάτων

Συγκρίνοντας τις αποδόσεις επιτυχημένης και αποτυχημένης αναζήτησης κάθε δομής δεδομένων, ερχόμαστε στο συμπέρασμα πως η διαφορά του βάθους ανάμεσα σε επιτυχημένη και αποτυχημένη αναζήτηση μεταξύ μιάς δομής για συγκεκριμένο M είναι ελάχιστη. Αυτό είναι αναμενόμενο αφού γνωρίζουμε ήδη πως το μεγαλύτερο ποσοστό των κλειδιών που εισάγουμε θα βρίσκεται στα τελευταία επίπεδα του κάθε δέντρου. Επιπλέον, είναι εμφανές και αναμενόμενο πως το μέσο βάθος που φτάνουμε στο QuadTree είναι πολύ μικρότερο από αυτό στο kdTree για ίδιο M. Παρατηρώντας τα διαγράμματα καταλήγουμε στο ότι η έκφραση πολυπλοκότητας που εκφράζει τις καμπύλες που σχεδιάσαμε είναι η : $O \in logM$. Αυτό επαληθεύεται και θεωρητικά , αφού γνωρίζουμε πως το Kd-Tree έχει πολυπλοκότητα log_2M και το QuadTree log_4M. Επιπλέον, βλέπουμε πως υπάρχει απόκλιση μεταξύ θεωρητικών και πειραματικών αποτελεσμάτων γεγονός το οποίο είναι αναμενόμενο, αφού δεν μπορούμε να εξασφαλίσουμε ισορροπημένα δέντρα.

Σχετικά με το ποιά δομή είναι πιο γρήγορη ελέγχουμε τους χρόνους που εκτυπώνουμε για τα 100000 κλειδιά(κοιτάμε για μεγάλο M , καθώς εκεί έχει περισσότερη σημασία) και βλέπουμε ότι στο Kd-Tree ο μέσος χρόνος για επιτυχημένη αναζήτηση είναι 1158ns και για αποτυχημένη είναι 1725ns. Αντίστοιχα, για το QuadTree ο μέσος χρόνος για επιτυχημένη αναζήτηση είναι 1725ns και για αποτυχημένη είναι 1278ns. Αρα , το KdTree είναι πιθανότατα πιο γρήγορο. Ωστόσο, για 70000 κλειδιά τυχαίνει το QuadTree να είναι γρηγορότερο. Ας αναλύσουμε λίγο το θέμα θεωρητικά. Θα μετρήσουμε τον αριθμό των πράξεων που εκτελούνται στα κομμάτια κώδικα που αντιστοιχούν στο searching του κάθε δέντρου. Για το kdTree έχουμε στο περίπου 8 πράξεις * το βάθος του δέντρου (worst case scenario). Αντίστοιχα, για το QuadTree έχουμε στο περίπου 19 πράξεις * το βάθος του δέντρου (worst case scenario). Αρα για M = 100000 έχουμε συνολικά 8 * 23 = 184 πράξεις για το kdTree , ενώ για το QuadTree έχουμε συνολικά 19 * 10 = 190 πράξεις στο περίπου. Για 70000 κλειδία έχουμε 8*25 = 200 πράξεις για το kdTree , ενώ για το QuadTree έχουμε συνολικά 19 * 10 = 190 πράξεις στο περίπου.

Συνεπώς, καταλήγουμε στο συμπέρασμα πως είναι λογικό να μην μπορούμε να πούμε ξεκάθαρα ποια δομή είναι γρηγορότερη. Ωστόσο, βλέπουμε πως το QuadTree έχει μικρότερη απόκλιση στο βάθος από την θεωρητική του τιμή για μεγάλα M σε σύγκριση με το kdTree. Δηδαλή το 2ο είναι λιγότερο ισορροπημένο δέντρο από το 1ο. Θεωρητικά για 100000 π.χ θα έπρεπε το βάθος του kdTree να είναι περίπου ίσο με 16.6 , ενώ του άλλου δέντρου με 8.3. Για αυτά τα αποτελέσματα έχουμε 8.3* 19 = 157.7 πράξεις για το QuadTree και 8*16.6 = 132.8 πράξεις για το KdTree. Σε αυτή την περίπτωση, θα ήταν πιό γρήγορο το KdTree (και για 70000).

Αλεξίου Σταμάτιος